

Edge cases (an ambiguous routing decision) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Read both depths — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

```
# supervisor_routing_practice.py
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command

SPECIALISTS = {
    "research_agent": "Looks up facts and data relevant to the task.",
    "analysis_agent": "Interprets data and explains what it means.",
}

class RoutingState(TypedDict):
    task: str
    picked: str

def supervisor(state):
    lines = []
    for name, desc in SPECIALISTS.items():
        lines.append(f"- {name}: {desc}")
    descriptions = "\n".join(lines)
    prompt = (
        f"Task: {state['task']}\n\n"
        "Choose exactly one specialist to handle this first:\n"
        f"{descriptions}\n\n"
        "Reply with only the specialist's name."
    )
```

```

response = model.invoke(prompt)
picked = response.content.strip()
return Command(update={"picked": picked}, goto=END)

builder = StateGraph(RoutingState)
builder.add_node("supervisor", supervisor)
builder.add_edge(START, "supervisor")
graph = builder.compile()

test_prompts = [
    "find out why sales dropped last quarter",
    "what does this survey data actually tell us",
    "look into the causes of the outage",
    "explain what happened to our conversion rate",
    "dig into the reasons behind the delay",
]

for prompt in test_prompts:
    result = graph.invoke({"task": prompt, "picked": ""})
    print(f"{prompt!r} -> {result['picked']}")

```

Expected output (LLM picks vary — this is one real run):

```

'find out why sales dropped last quarter' -> research_agent
'what does this survey data actually tell us' -> analysis_agent
'look into the causes of the outage' -> research_agent
'explain what happened to our conversion rate' -> analysis_agent
'dig into the reasons behind the delay' -> research_agent

```

This works and shows real routing decisions on genuinely ambiguous prompts. It's missing repeated runs to check consistency, and it has no way to recover if the pick turns out to be the less useful one — both fine for a first pass at seeing the ambiguity happen at all.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — repeated runs, to check routing consistency

```
# supervisor_routing_practice.py
def run_ambiguity_check(prompts: list[str], attempts: int = 3) -> dict:
    tally = {}
    for prompt in prompts:
        picks = []
        for _ in range(attempts):
            result = graph.invoke({"task": prompt, "picked": ""})
            picks.append(result["picked"])
        tally[prompt] = picks
    return tally

if __name__ == "__main__":
    test_prompts = [
        "find out why sales dropped last quarter",
        "what does this survey data actually tell us",
        "look into the causes of the outage",
        "explain what happened to our conversion rate",
        "dig into the reasons behind the delay",
    ]
    tally = run_ambiguity_check(test_prompts)
    for prompt, picks in tally.items():
        consistent = len(set(picks)) == 1
        label = "consistent"
        if not consistent:
            label = "INCONSISTENT"
        print(f"{prompt!r}:")
        print(f"    {picks} ({label})")
```

Expected output (exact picks vary by run):

```
'find out why sales dropped last quarter':
    ['research_agent', 'research_agent', 'analysis_agent'] (INCONSISTENT)
'what does this survey data actually tell us':
    ['analysis_agent', 'analysis_agent', 'analysis_agent'] (consistent)
'look into the causes of the outage':
```

```

['research_agent', 'research_agent', 'research_agent'] (consistent)
'explain what happened to our conversion rate':
['analysis_agent', 'research_agent', 'analysis_agent'] (INCONSISTENT)
'dig into the reasons behind the delay':
['research_agent', 'research_agent', 'research_agent'] (consistent)

```

Difference from Basic: the same routing call, run 3 times per prompt instead of once, which is the only way to tell a genuinely stable routing decision (`consistent`) apart from one where the model is essentially coin-flipping on a truly ambiguous prompt (`INCONSISTENT`). Notice the prompts written with the strongest "why/what happened" framing (implying analysis) turned out to be the least consistent ones — a useful, concrete finding this version surfaces that a single run never would. Still no recovery path if the pick was the less useful one — that's what Approach 2 adds.

Approach 2 — a guard in each specialist, and a supervisor that can recover from a misroute

```

# supervisor_routing_practice.py
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command

class RoutingState(TypedDict):
    task: str
    picked_history: list
    research_data: str
    analysis: str
    misrouted: bool

def supervisor(state: RoutingState) -> Command:
    if state.get("analysis"):
        return Command(goto=END)

    descriptions = (
        "- research_agent: Looks up facts and data relevant to the task.\n"
        "- analysis_agent: Interprets data and explains what it means."
    )
    prompt = (
        f"Task: {state['task']}\n\n"
        "Choose exactly one specialist to handle this first:\n"
        f"{descriptions}\n\n"
        "Reply with only the specialist's name."
    )
    response = model.invoke(prompt)

```

```

        picked = response.content.strip()
        return Command(
            update={"picked_history": state["picked_history"] + [picked]},
            goto=picked,
        )

def research_agent(state: RoutingState) -> Command:
    state_research = f"[research data for: {state['task']}]"
    if state.get("misrouted"):
        # this run started as a recovered misroute -- go straight to
        # analysis now
        return Command(
            update={"research_data": state_research, "misrouted": False},
            goto="analysis_agent",
        )
    return Command(update={"research_data": state_research}, goto=END)

def analysis_agent(state: RoutingState) -> Command:
    if not state.get("research_data"):
        # picked first, but nothing to analyze yet -- signal instead of
        # forging ahead
        history = state["picked_history"] + ["[recovered->research_agent]"]
        return Command(
            update={"misrouted": True, "picked_history": history},
            goto="research_agent",
        )
    analysis = f"[analysis of: {state['research_data']}]"
    return Command(update={"analysis": analysis}, goto=END)

builder = StateGraph(RoutingState)
builder.add_node("supervisor", supervisor)
builder.add_node("research_agent", research_agent)
builder.add_node("analysis_agent", analysis_agent)
builder.add_edge(START, "supervisor")
graph = builder.compile()

```

```

# supervisor_routing_practice.py
def run_full_tally(prompts: list[str], attempts: int = 3) -> None:
    first_pick_counts = {"research_agent": 0, "analysis_agent": 0}
    recovered_count = 0
    total_runs = 0

    for prompt in prompts:

```

```

    for _ in range(attempts):
        result = graph.invoke({
            "task": prompt, "picked_history": [], "research_data": "",
            "analysis": "", "misrouted": False,
        })
        total_runs += 1
        first_pick = result["picked_history"][0]
        first_pick_counts[first_pick] += 1
        if "[recovered->research_agent]" in result["picked_history"]:
            recovered_count += 1

    research_first = first_pick_counts["research_agent"]
    analysis_first = first_pick_counts["analysis_agent"]

    print(f"total runs: {total_runs}")
    print(f"first pick -- research: {research_first}")
    print(f"first pick -- analysis: {analysis_first}")
    print(f"misroutes recovered: {recovered_count}")

test_prompts = [
    "find out why sales dropped last quarter",
    "what does this survey data actually tell us",
    "look into the causes of the outage",
    "explain what happened to our conversion rate",
    "dig into the reasons behind the delay",
]
run_full_tally(test_prompts)

```

Expected output (exact numbers vary by run):

```

total runs: 15
first pick -- research: 9
first pick -- analysis: 6
misroutes recovered: 6

```

Every time `analysis_agent` was picked first, it had nothing to analyze yet and had to recover through `research_agent` — meaning `analysis_agent` should arguably never be the correct first pick for these particular prompts, even though the model chose it 6 times out of 15. That's the real finding this exercise is built to produce: the supervisor's routing description for `analysis_agent` needs to say more clearly that it requires research data first, or the two descriptions need to stop overlapping so closely.

Difference between Approach 1 and Approach 2: Approach 1's repeated runs tell you *that* routing is inconsistent on some prompts, but a misroute there is still a dead end — nothing catches it.

Approach 2 adds the guard (`analysis_agent` checking for `research_data` before doing any real work) and the recovery path (routing back to `research_agent` , then forward again), so a misroute becomes a visible, logged, recoverable extra hop instead of a confident answer built on nothing — directly applying Core Concepts' Error propagation guidance ("look at what an agent actually returned, not just assume it worked because no exception was thrown") one level down, at the routing layer itself.

Which one should you actually write? Basic is enough to see that two specialist descriptions overlap in practice — worth doing once, early, whenever you're unsure if two specialists are too similar. Approach 1's repeated-run consistency check is worth running on every supervisor before it ships, since a routing decision that flips on identical input is a real bug waiting to surface intermittently in production. Approach 2's guard-and-recover pattern is the one to actually keep in the code: cheap to add (one `if` per specialist checking its own prerequisites), and it converts the single most common supervisor-pattern production failure — routing to a specialist that isn't ready yet — from a silent bad answer into a self-correcting extra step, which is exactly the resilience Project 4's own Research-vs-Analysis boundary needs.